Trabajo práctico 6.2
BST - AVL

BST

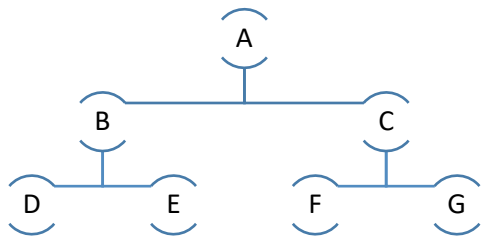
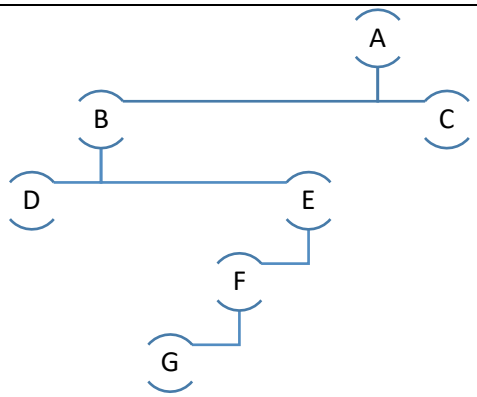
1. Implemente una función `es_BST(T)` en Python que dado un árbol binario T retorne True si es BST o False en caso contrario.
2. Implemente la función `buscar(T,x)` en Python que, dado un árbol binario T y un valor x, indique si x pertenece o no a T.
3. Implemente la función `insertar(T,x)` en Python que, dado un árbol binario T y un valor x, inserte correctamente x en T.
4. Implemente una función `buscar2(T,x)` en Python que, dado un árbol binario T y un valor x, retorne una tupla (p, a)

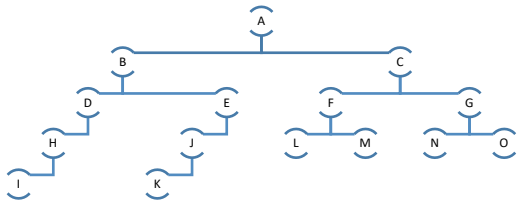
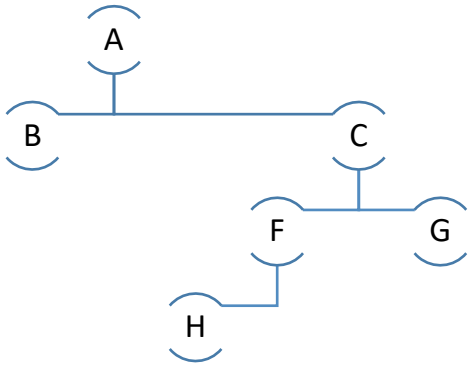
   donde:
   a es el nodo que tiene valor x
   p es el padre de a

5. Implemente la función `eliminar(T,x)` en Python que, dado un árbol binario T y un valor x, elimine x de T.
6. Implemente completamente la clase BST que incluye las operaciones anteriores.
7. Incorpore a la clase BST el método `__str__(self)`. El mismo debe retornar una cadena que permita ver en pantalla la estructura del árbol.

AVL

8. Escriba una definición recursiva de un árbol AVL.
9. Implemente una función `es_AVL(T)` que, dado un árbol BST T, devuelva True si T respeta la definición de AVL o Falso en caso contrario.
10. Dado los siguientes árboles binarios indique para cada nodo
    a. Altura del subárbol que lo tiene por raíz
    b. Factor de balance

a.-	**b.-**
	

c.-	d.-
	

11. Dibuje el árbol luego de cada operación (considere las rotaciones).

a.	b.

`a = AVL()`
`a.insert(2)`
`a.insert(4)`
`a.insert(8)`
`a.insert(10)`
`a.insert(12)`
`a.insert(100)`
	`a = AVL()`
	`a.insert(100)`
	`a.insert(12)`
	`a.insert(10)`
	`a.insert(8)`
	`a.insert(4)`
	`a.insert(2)`

c.	d.

`a = AVL()`
`a.insert(2)`
`a.insert(4)`
`a.insert(8)`
`a.insert(10)`
`a.remove(10)`
`a.remove(8)`
	`a = AVL()`
	`a.insert(50)`
	`a.insert(100)`
	`a.insert(40)`
	`a.insert(200)`
	`a.remove(50)`